



CS-361 Project 2

Comparing Array vs Priority Queue Implementations of Dijkstra's Algorithm

May 5, 2026

Andrew Crowe
Nathan Stallman

Table of Contents

Table of Contents.....	2
GitHub Repo.....	3
4.1 Introduction.....	3
4.2 Implementation.....	4
4.3 Graph Types.....	4
4.4 Experiment Results.....	5
4.5 Result Analysis.....	8
Analysis and Theory.....	8
Discussion.....	9
1. Did both implementations always produce the same shortest path distances?..	9
2. On which graph types was the priority queue implementation clearly faster?...	9
3. Was there any graph size where the array implementation was still competitive?.....	10
4. How closely did the measurements follow the theoretical complexity?.....	10
5. What practical factors may explain unexpected results?.....	10
Conclusion.....	10
Additional Notes.....	11
1. Prim's Algorithm Usage.....	11
Distribution of work.....	12
Bibliography.....	12

GitHub Repo

<https://github.com/andrew-crowe-ok/dijkstra-array-vs-heap>

4.1 Introduction

The shortest path problem asks: What is the shortest path from a starting location to a destination? Edsger Dijkstra developed a solution while pondering the same question, in the context of travelling efficiently from city to city through the Netherlands. Today, variations of Dijkstra's algorithm form the backbone of services such as GPS navigation, network traffic handling, and social media networks, just to name a few.

Any problem that requires a single source shortest path solution is a problem where Dijkstra is likely applicable, with one caveat. The edges of the graph must not be negatively-weighted. This is because Dijkstra's algorithm operates on an assumption that the sum of the weights of any path through a graph will always be greater than or equal to the number of edges on that path. This assumption underlies the algorithm's greedy nature; it can choose the locally optimal solution at every point and still guarantee that it will find the globally optimal solution by the end. The possibility of negatively weighted edges breaks this property. Additionally, this underlying assumption means that any cycle where the sum of its edges is negative will cause Dijkstra to get caught in an infinite loop.

There are two mechanisms in Dijkstra's algorithm that are responsible for its failure with negative weights. However, they are also at the core of the algorithm's efficiency in appropriate use cases. As it traverses the graph, it tracks the shortest known path to every node it has encountered so far. As each new node is visited, new information is gained and the shortest known paths are updated if shorter paths are discovered. This process is known as relaxation, and it ensures that the algorithm has the information necessary to choose the optimal solution at every node. Another piece of choosing the optimal solution is the data structure that manages traversal. Implemented as a min-heap, Dijkstra utilizes a priority queue to store unvisited nodes. Every time the algorithm is ready to select a new node, it pops the queue and receives the node at the minimum distance from the currently selected node.

4.2 Implementation

Dijkstra's were created in Python3, the program constructs both types of adjacency graphs and runs them through the two different implementations of Dijkstra's based on what the user chooses to run via the command line. The array based Dijkstra is a basic algorithm simply using arrays to check what nodes were visited and determine ideal distances whereas the priority queue uses a min heap data structure to speed up the process at scale. R code was used to generate graphs

4.3 Graph Types

Adjacency lists and matrices were built taking arrays with tuples and generating the respective nodes and weights in the graphs. For the third dense graph a random generator was created, however the first spanning tree version did not create graphs that were actually dense, the hope was the sheer number of edges would make the graph dense however the opposite relation was found as it got sparser from 12 nodes to 18 nodes. This prompted a full rewrite of the random function, this time using Prim's Algorithm as it could reuse the priority queue and as it is a greedy algorithm that runs similar to Dijkstra some of that code could be used as well (see **Additional Notes 1.** for more info). Finally, another custom graph was written this time measuring distances to cities within New Mexico, this shows a practical use of Dijkstra well calling back to its inception of Edsger W. Dijkstra himself noticing the relation in Amsterdam.

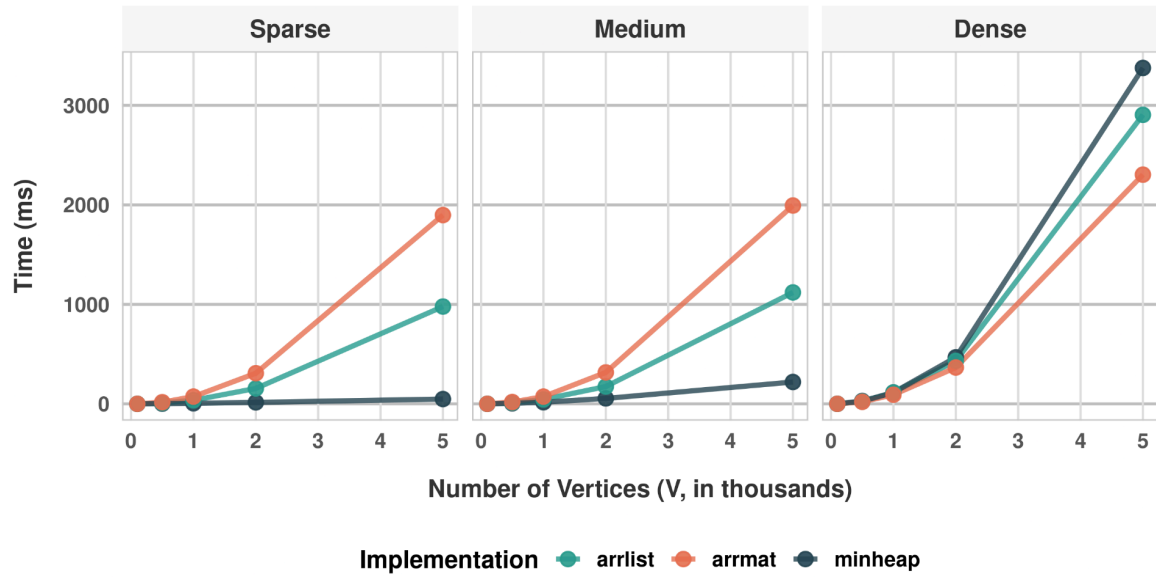
4.4 Experiment Results

Graph	V	E	Source	Array (adj list)	Array (adj matrix)	Min-heap (PQ)	Match?
Sparse graph one	6	6	A	0.000011 s	0.000011 s	0.000018 s	Yes
Sparse graph two	7	7	1	0.000007 s	0.000007 s	0.000016 s	Yes
Dense graph one	5	10	A	0.000006 s	0.000005 s	0.000013 s	Yes
Dense graph two	6	15	1	0.000006 s	0.000006 s	0.000015 s	Yes
Dynamic Dense Graph	18	107	0	0.000022 s	0.000028 s	0.000055 s	Yes
New Mexico Graph	8	15	Abq	0.000007 s	0.000009 s	0.000019 s	Yes

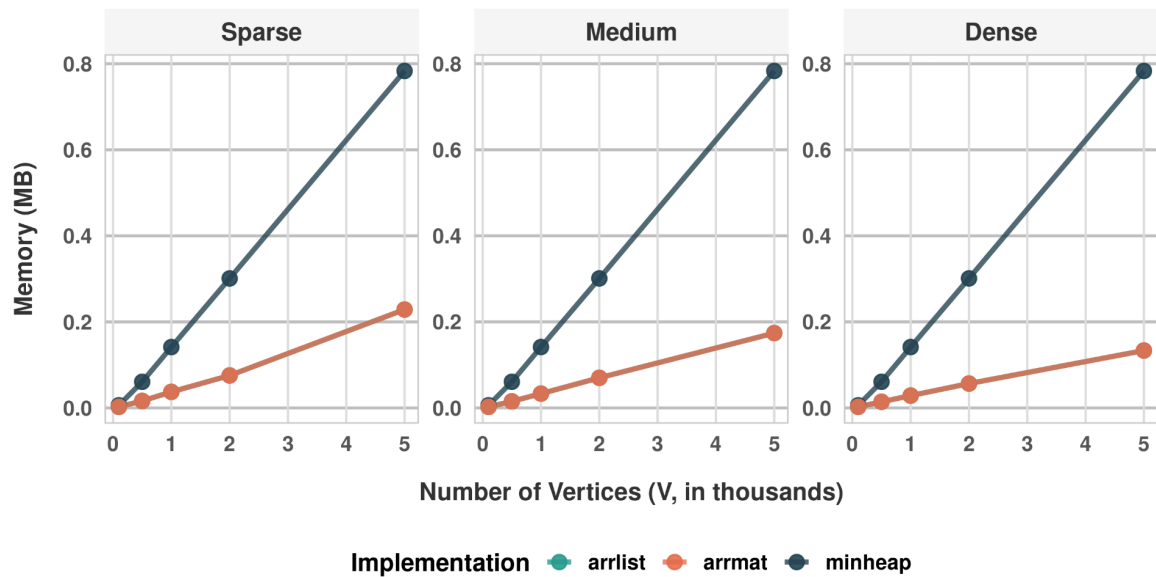
Graph #	Graph type	Method	Time (ms)	Memory (KB)
1	Sparse-1 6V · 6E · 40%	Array (adj. list)	0.031	0.28
		Array (adj. matrix)	0.034	0.28
		Heap (adj. list)	0.038	0.70
2	Sparse-2 7V · 7E · 33.3%	Array (adj. list)	0.015	0.30
		Array (adj. matrix)	0.022	0.30
		Heap (adj. list)	0.028	0.79
3	Dense-1 5V · 10E · 100%	Array (adj. list)	0.012	0.26
		Array (adj. matrix)	0.018	0.26
		Heap (adj. list)	0.024	0.59
4	Dense-2 6V · 15E · 100%	Array (adj. list)	0.013	0.28
		Array (adj. matrix)	0.015	0.28
		Heap (adj. list)	0.026	0.62
5	Dense-3 18V · 107E · 69.9%	Array (adj. list)	0.037	0.56
		Array (adj. matrix)	0.047	0.56
		Heap (adj. list)	0.077	1.92
6	Dense-4 8V · 15E · 53.6% (NM)	Array (adj. list)	0.016	0.33
		Array (adj. matrix)	0.019	0.33
		Heap (adj. list)	0.030	0.69

For better measurements of time and data the large data sets in the CSVs were also used to ensure the trends seen in this table continued at scale. The results of which can be seen in the graphs on the next page. The example being from an ASUS Windows computer (nathan_asus_win_benchmark_results.csv) but all the data sets follow the same trend

Dijkstra's Algorithm: Execution Time



Dijkstra's Algorithm: Peak Memory Usage



4.5 Result Analysis

The graphs ran generally as expected across both Dijkstra implementations. Basic arrays are more memory efficient and have significantly less overhead making them better at smaller edges but they scale extremely poorly running at $O(V^2)$. This is where we see the priority queue min heap show its benefit as the overhead becomes justified for the time saving. Even in the small dense test graphs with 10+ nodes you see the priority queue saving a few seconds of time, array is quickly outpaced. In our testing the array version always matched the priority queue version, this does make sense as they share much of the same code as they have the exact same setup the only real difference is the use of the priority queue vs arrays, as in the array version the visited nodes and distances are instead relegated to your typical array function. Overall, once everything was working properly both Dijkstra implementations performed exactly how we expected

Analysis and Theory

Dijkstra's algorithm varies in time complexity depending on the input size and underlying implementation of the priority queue. Array-based implementations are $O(V^2)$ - under the worst circumstances, they will need to scan V vertices V times to identify the next-minimum-distance vertex. The heap implementation is $O((V + E) \log V)$. It takes advantage of the logarithmic nature of a heap to achieve efficient selection of the next-minimum-distance vertex. Heapification alone is $O(\log V)$, and it will need to be performed at most $V + E$ times.

Our experiments demonstrate that, despite the superior time complexity of the heap implementation of Dijkstra, there is a point where it is no longer the optimal data structure. In dense graphs, $V + E$ can become large enough that its scalar effect on $O(\log V)$ heapification causes the growth rate to exceed $O(V^2)$. With respect to space complexity, graph density had no measurable impact on growth rates. In all scenarios, the heap structure exhibits a roughly $O(V)$ growth rate, performing significantly worse than the approximately logarithmic growth rate of the array-based implementations.

Our experiments show that the heap implementation performs best for non-dense graphs. It achieves its expected $O((V + E) \log V)$ time complexity and significantly outperforms the $O(V^2)$ complexity of the array-based implementations. As before, the growth rate is static across all graph densities, with the arrays performing significantly better than the heap.

Our experimental data supports theoretical expectations. The data recorded for our sample graphs that appears to be inconsistent with theory can be safely ignored. The small input sizes of those graphs means that other variables, such as the specific machine hardware or software optimizations (among many, many other possible variables) have far more impact on the absolute running time. After properly benchmarking our algorithms by taking the average of many runs over much larger input sizes, the data reveals a clear trend that closely follows theoretical expectations.

The goal of this experiment was to investigate the impact of different implementations of a priority queue on the time and space complexity of Dijkstra's single source shortest path algorithm. The job of the priority queue is to select the next-minimum-distance vertex during graph traversal. Our experiment confirmed that a minheap built with an adjacency list is the ideal implementation in most circumstances. However, both array-based implementations beat heaps in dense graphs, with the array + adjacency matrix construction performing best in that scenario. For non-dense graphs, the array + adjacency list construction was the more efficient array-based implementation. Both array structures performed identically in space complexity, and substantially better than the heap. Our experiments yielded results that largely confirm theoretical expectations of the performance of Dijkstra's algorithm when it is built from different priority queue structures.

Discussion

1. Did both implementations always produce the same shortest path distances?

Both implementations always produce the same shortest path distance because Dijkstra is designed such that it always finds the exact shortest path. Other algorithms, such as a generic depth-first search, could return a different path length because they are not actually designed to find the shortest path.

2. On which graph types was the priority queue implementation clearly faster?

Our data shows that the heap priority queue performs best on non-dense graphs. The heap structure's strengths are most apparent here.

3. Was there any graph size where the array implementation was still competitive?

According to our data plots, there was no circumstance in which the array implementation was “competitive.” It either performs significantly worse than the heap, or it outperforms the heap past a certain graph density.

4. How closely did the measurements follow the theoretical complexity?

Aside from the data recorded on sample graphs, measurements followed theoretical complexity nearly perfectly.

5. What practical factors may explain unexpected results?

In real world testing there are many factors that can impact results as programs do not run in a vacuum, they run on complex computers often running hundreds of other tasks simultaneously. While benchmarking we made sure our results were consistent on our hardware and were in line with how the algorithms should operate. Some odd results came up with the old randomly generated spanning tree graphs but these were fixed once it was made into a more proper generator with Prim's Algorithm and run with less randomness.

Conclusion

Dijkstra's algorithm is an extremely foundational algorithm in computer science, being so good at what it does, its dominance in the industry has been largely unchallenged in 60 years. What this project has shown is why and how Dijkstra is an effective algorithm, functionally it is simple to understand and where implementing it is not quite as simple it is an extremely versatile algorithm being able to be ran with simple arrays, more advanced priority queue min heaps, and with adjacency lists and matrices. This flexibility means there are many ways to optimize it to your specific use case, this along with its efficiency is what has made it a power house of an algorithm for the last half a century.

Additional Notes

This is for information about additional parts of the codebase not required by the project and final thoughts on the program

1. Prim's Algorithm Usage

Prim's Algorithm is used to implement the random dense graph, this came about originally as a basic spanning tree generator. This however was not capable of creating true dense graphs, so this led to research into a new generator. Prim's algorithm was chosen for this as it focuses on edge weight and is used for MSTs which naturally avoid self loops and duplicate edges which is required to make a dense graph, eliminating cycles was the goal of the original spanning tree implementation. The other direct benefit of Prim's algorithm is its strong similarity to Dijkstra, both are greedy algorithms that measure the weights of graphs. This meant that the min heap priority queue could get more use and even the array implementation of Dijkstra could be used to construct the algorithm used to generate the graphs. For use in the program it first grows a MST starting at node 0 and rolls the weights using Python's built in random function. The randomness was scaled back in that most of the values were hard and the runs were seeded to 37, this was because the function was created to better understand the Dijkstra's core functions and it was never tested with edge cases

Distribution of work

- Introduction/background draft complete - *Andrew*
- Adjacency list and matrix implemented - *Nathan*
- Randomly generated dense graph - *Nathan*
- New Mexico dense graph - *Andrew*
- Min-heap priority queue implemented - *Andrew*
- PQ Dijkstra - *Andrew*
- Arr Dijkstra - *Nathan*
- General repo organization - *Andrew*
- Document formatting - *Nathan*

Bibliography

GeeksforGeeks. (2025, July 23). *Adjacency list representation*.

<https://www.geeksforgeeks.org/dsa/adjacency-list-meaning-definition-in-dsa/>

GeeksforGeeks. (n.d.). *Adjacency matrix*.

<https://www.geeksforgeeks.org/dsa/adjacency-matrix/>

GeeksforGeeks. (n.d.). *Prim's minimum spanning tree (MST) | Greedy algorithm*.

<https://www.geeksforgeeks.org/dsa/prims-minimum-spanning-tree-mst-greedy-algo-5/>